

Big Data Programming

Basic Operators and Control Statement

Chapter Overview

Oveview

실행 환경을 구분한다

CLI와 Python 대화형 모드의 차이를 확인한다.
같은 Python 코드라도 스크립트와 Notebook에서는
코드를 실행하고 결과를 관리하는 방식이 다르다.

입력한 문자열을 계산에 사용한다

input()으로 받은 값의 자료형을 확인하고 int()와 float()로 변환한다.
입력, 계산, 출력을 나누어 읽으면 오류가 생긴 위치를 찾기 쉽다.

기본 연산자를 목적에 맞게 선택한다

산술·비교·논리·멤버십·비트 연산을 구분한다.
화씨 온도 변환 문제를 해결하고, 같은 기호라도 값의 자료형과 연산 순서가 중요함을 확인한다.

Command Line Interface

GUI와 CLI는 명령을 전달하는 방식이다

사용자 인터페이스의 의미

사용자 인터페이스는 사람이 프로그램에 명령을 전달하고 결과를 확인하는 접점이다. GUI는 창·메뉴·버튼을, CLI는 입력한 명령문을 중심으로 사용한다.

VS Code에서 두 방식을 함께 사용한다

파일을 여는 메뉴와 실행 버튼은 GUI의 예이다. 통합 터미널에서 python hello.py를 입력하는 것은 CLI의 예이다. 두 방식 모두 같은 파일을 실행할 수 있다.

명령문은 작업 순서를 남긴다

사용한 명령과 파일명을 기록하면 실행 과정을 다시 따라갈 수 있다. 단, 현재 폴더나 선택한 Python이 달라지면 같은 명령도 다른 결과를 낼 수 있다.

터미널에서 실행하는 예

python hello.py

확인할 세 가지

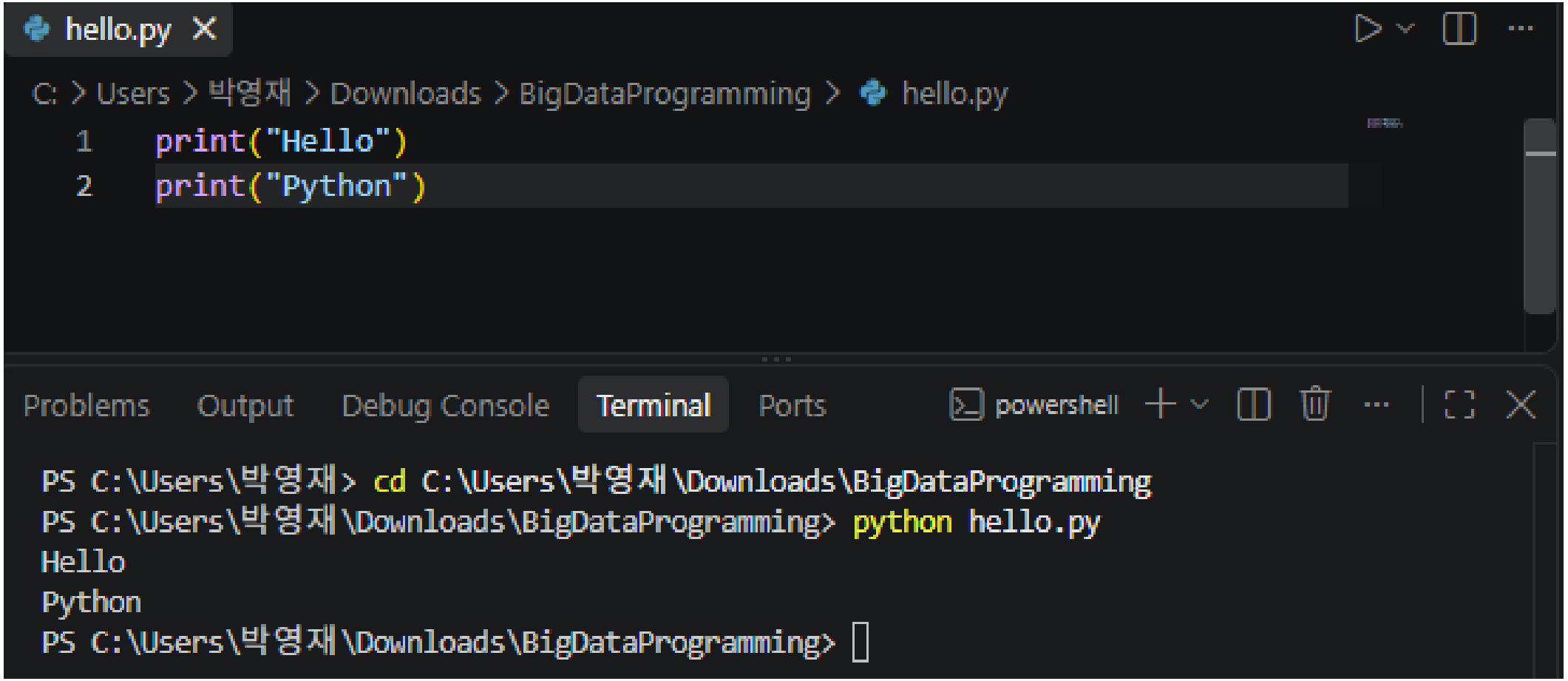
실행할 파일의 이름, 현재 작업 폴더, 코드를 실행하는 Python 환경을 확인한다.

GUI와 CLI의 관계

CLI가 GUI보다 항상 우수한 것은 아니다. 작업에 맞게 버튼과 명령문을 함께 사용한다.

Command Line Interface

VS Code의 편집 화면과 터미널을 함께 읽기



위쪽은 hello.py를 작성하는 편집 영역이고, 아래쪽은 명령을 입력하는 터미널이다.
python hello.py가 실행된 뒤 Hello와 Python이 순서대로 출력된다.

Command Line Interface

터미널 명령과 Python 코드를 구분하기

운영체제의 셸이 받는 명령

python hello.py는 Python을 시작하여 파일을 실행하라는 셸 명령이다.
.py 파일 본문에 그대로 적는 Python 문장이 아니다.

셸에 입력

```
python hello.py
python
```

Python 대화형 모드가 받는 코드

터미널에서 python을 실행하면 Python 대화형 모드에 들어갈 수 있다. >>> 뒤에는 print()나 계산식처럼 Python 문법으로 작성한 코드를 입력한다.

Python 안에 입력

```
print(3 + 4)
exit()
```

입력 위치부터 확인한다

명령을 잘못된 창에 넣으면 문법 오류가 날 수 있다.
>>>는 안내 기호이므로 코드에 복사하지 않는다.
대화형 모드를 끝내려면 exit()를 사용한다.

명령 이름은 환경마다 다를 수 있다

Windows에서는 py, macOS에서는 python3를 사용할 수도 있다. 설치한 Python을 가리키는 명령인지 확인하고 수업 중 일관되게 사용한다.

Command Line Interface

셸과 Python 대화형 모드의 프롬프트 구분하기

```
$ python hello.py
Hello
Python
$ python -q
>>> 3 + 4
7
>>> print("Hello")
Hello
>>> exit()
$
```

이 Linux 터미널 예시에서 $ 뒤에는 셸 명령을, >>> 뒤에는 Python 코드를 입력한다.
exit()를 실행하면 셸로 돌아온다. python -q는 시작 안내 문구를 생략한다.

Standard Input and Output

입력 → 처리 → 출력으로 코드 읽기

입력은 프로그램 밖의 값을 받는다

input()은 사용자가 입력을 마칠 때까지 기다린다. 콘솔에서 Enter를 누르면 입력한 한 줄을 문자열로 받아 다음 문장으로 진행한다.

처리는 받은 값으로 일을 수행한다

자료형 변환, 계산, 조건 판단이 처리에 해당한다. 어떤연산을 할 것인지에 따라 필요한 자료형이 달라지므로 입력과 처리를 구분해야 한다.

출력은 결과를 확인하게 한다

print()는 전달한 값을 출력한다. 여기서는 터미널과 Notebook의 출력 영역을 사용하며, 일반적인 콘솔 입출력만 다룬다.

이름을 입력받아 인사하기

```
name = input("이름: ")
print("안녕하세요,", name)
```

입력 예와 출력

이름: 영재
안녕하세요, 영재

예제 해석

영재는 사용자가 입력한 값이다. 프로그램은 name에 연결된 문자열을 출력할 때 다시 사용한다.

Jupyter Notebook

Notebook은 셀 단위로 실행하는 문서이다

코드와 설명을 같은 문서에 둔다

Notebook 파일의 확장자는 .ipynb이다. 코드 셀에는 Python 코드를, Markdown 셀에는 설명을 작성한다. 실행 결과가 코드와 함께 남아 계산 과정을 살펴보기 편리하다.

한 셀에는 여러 줄을 작성할 수 있다

실행 단위는 한 줄이 아니라 하나의 셀이다. 셀 안에서는 코드가 순서대로 실행된다. 셀을 나눈다고 서로 다른 Python 환경이 되는 것은 아니다.

편집 화면과 커널을 구분한다

Notebook은 VS Code나 웹브라우저에서 열 수 있다. 실제 코드를 실행하고 변수 상태를 유지하는 프로세스를 커널(kernel)이라고 한다.

첫 번째 코드 셀

```
x = 10
y = 3
print(x + y)
```

셀 아래에 나타나는 결과

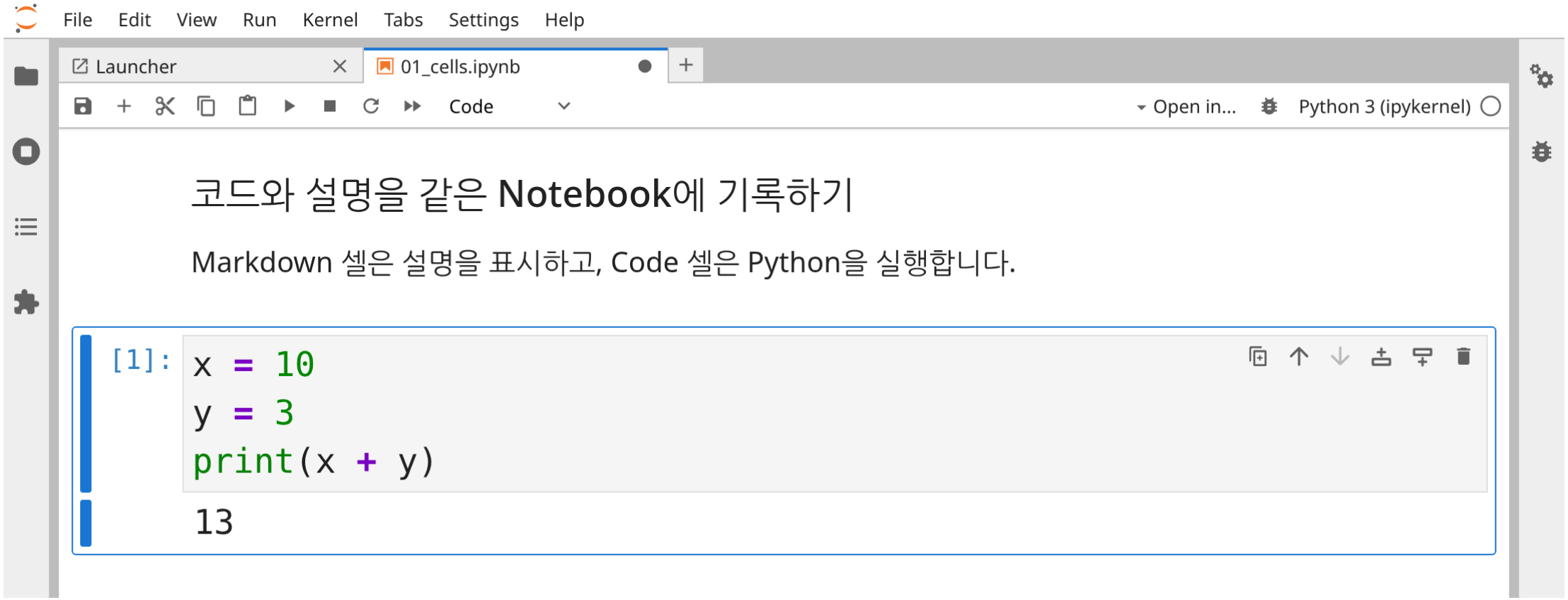
13

설명 셀의 예

“두 수를 더한 결과를 확인한다.”와 같은 문장은 Markdown 셀에 적는다. 설명 셀은 Python 계산을 실행하는 코드 셀과 다르다.

Jupyter Notebook

코드 셀, 설명 셀, 출력 영역을 실제 화면에서 찾기



이후 Notebook 캡처는 브라우저용 JupyterLab 화면이다. 같은 .ipynb를 VS Code에서도 열 수 있다.
위쪽은 Markdown 설명이며, 아래 Code 셀의 세 줄을 실행하면 셀 아래에 13이 출력된다.

Jupyter Notebook

VS Code에서 Notebook 실행 환경 준비하기

기존 Python 환경을 사용한다

VS Code에서 수업 폴더를 열고 Microsoft의 Python 확장과 Jupyter 확장을 확인한다. 확장은 편집 기능을 제공하며, 실제 Python 실행 환경과는 구분된다.

선택한 환경에 커널을 준비한다

Notebook에 사용할 Python 환경의 터미널에서 ipykernel을 설치한다. python -m pip 형식은 해당 python 명령이 가리키는 환경에 패키지를 설치하도록 한다.

인터프리터와 커널 선택을 확인한다

스크립트의 인터프리터 선택과 Notebook의 커널 선택은 별도로 확인한다. 서로 다른 환경을 선택하면 설치한 패키지가 보이지 않을 수 있다.

터미널에서 확인하고 설치

```
python --version
python -m pip --version
python -m pip install notebook ipykernel
python -m notebook
```

VS Code 확장 검색

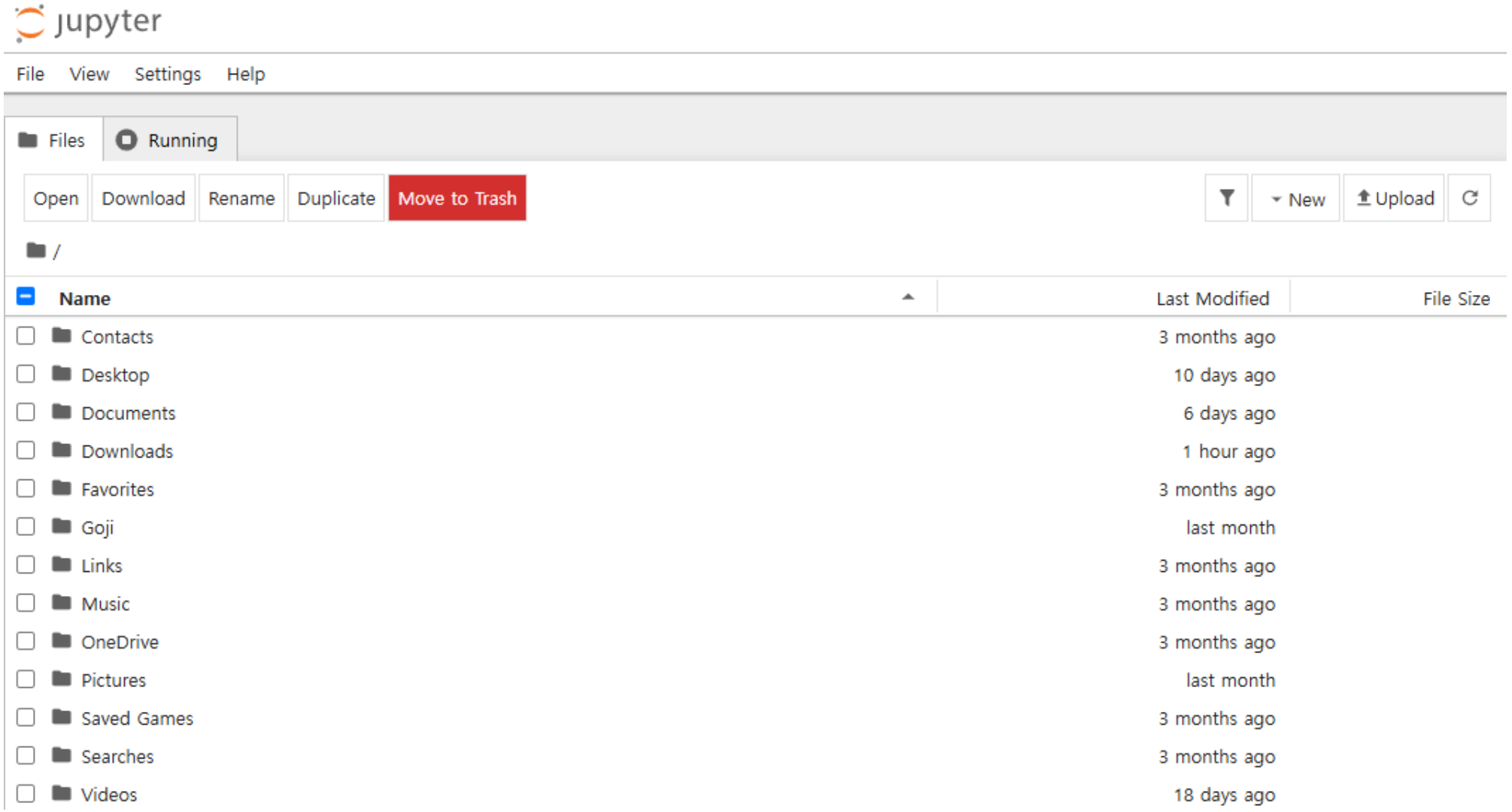
Python — Microsoft
Jupyter — Microsoft

설치할 환경이 맞는지 확인

여러 Python이 설치되어 있으면 버전뿐 아니라 실행 파일의 위치도 확인한다. 무작정 다른 명령으로 재설치하지 않는다.

Jupyter Notebook

VS Code에서 Notebook 실행 환경 준비하기



Jupyter Notebook

터미널에서 Python과 커널 패키지 확인하기

```
$ python --version
Python 3.13.5
$ python -m pip --version
pip 25.1.1 from /opt/pyvenv/lib/python3.13/site-packages/pip (python 3.13)
$ python -c "import ipykernel; print(ipykernel.__version__)"
7.2.0
$
```

설치 후 Python, pip, ipykernel의 버전을 확인한 실제 실행 예이다. 버전과 경로는 환경마다 다르다.
패키지를 설치한 Python과 Notebook에서 선택한 Python이 같은 환경인지 함께 확인한다.

Jupyter Notebook

Notebook 만들기와 커널 선택하기

새 Notebook을 만든다

명령 팔레트에서 Create: New Jupyter Notebook을 실행하거나 .ipynb 파일을 만든다. 파일은 수업 폴더에 ch2_practice.ipynb로 저장한다.

실행할 Python 커널을 지정한다

오른쪽 위 Select Kernel에서 수업에 사용할 Python 환경을 선택한다. 필요한 경우 Select Another Kernel에서 다른 환경 목록을 확인한다.

첫 실행 결과를 확인한다

코드 셀에서 print("Hello, Notebook!")를 실행한다. 셀 아래에 결과가 보이면 편집 문서와 실행 커널의 연결이 동작하는 것이다.

첫 번째 셀

```
print("Hello, Notebook!")
```

실행 결과

```
Hello, Notebook!
```

커널의 실행 파일 확인

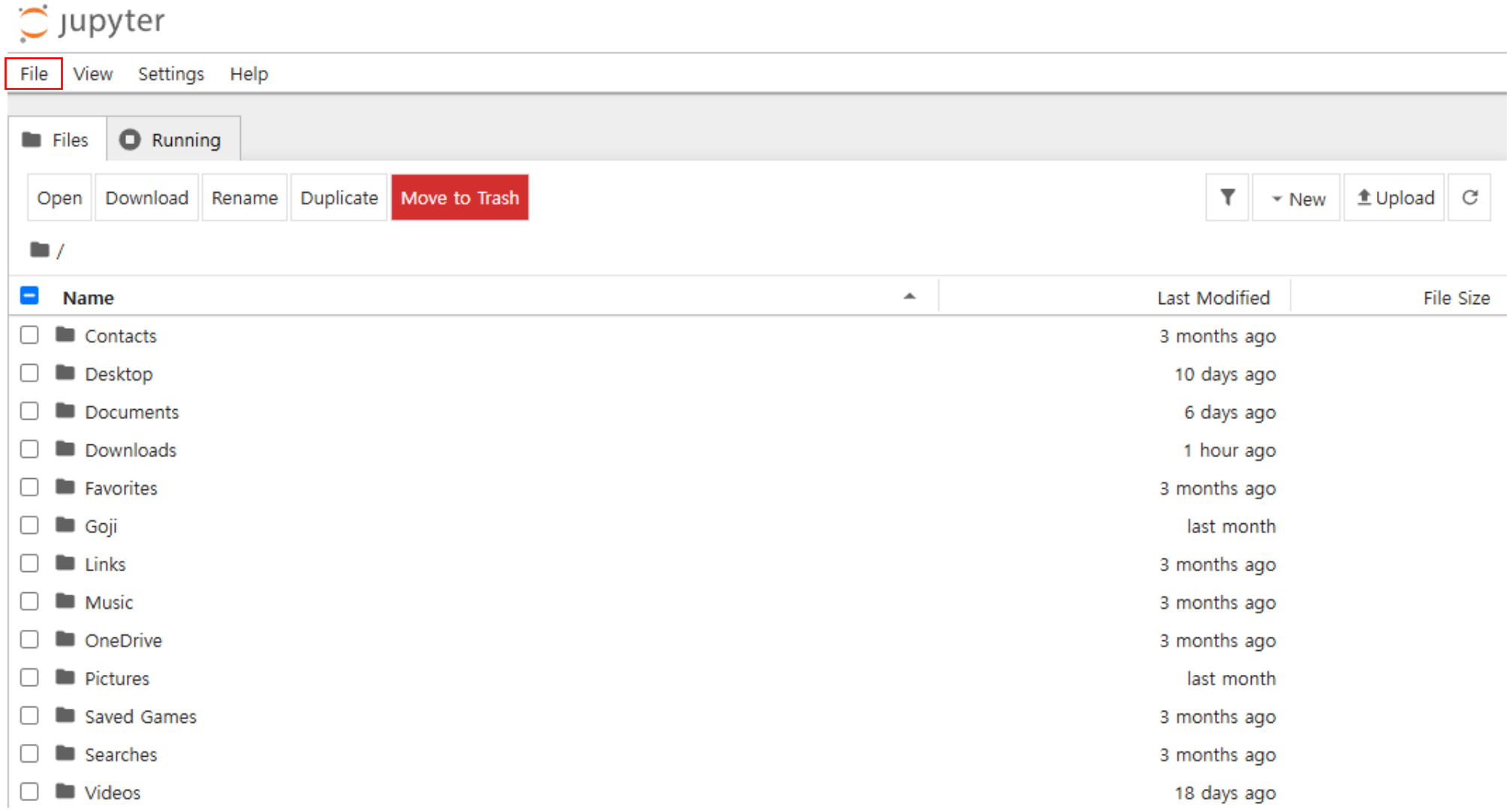
```
import sys
print(sys.executable)
```

확인할 내용

출력된 경로가 사용하려던 Python 환경인지 확인한다. 메뉴 위치는 VS Code 버전에 따라 조금 달라질 수 있다.

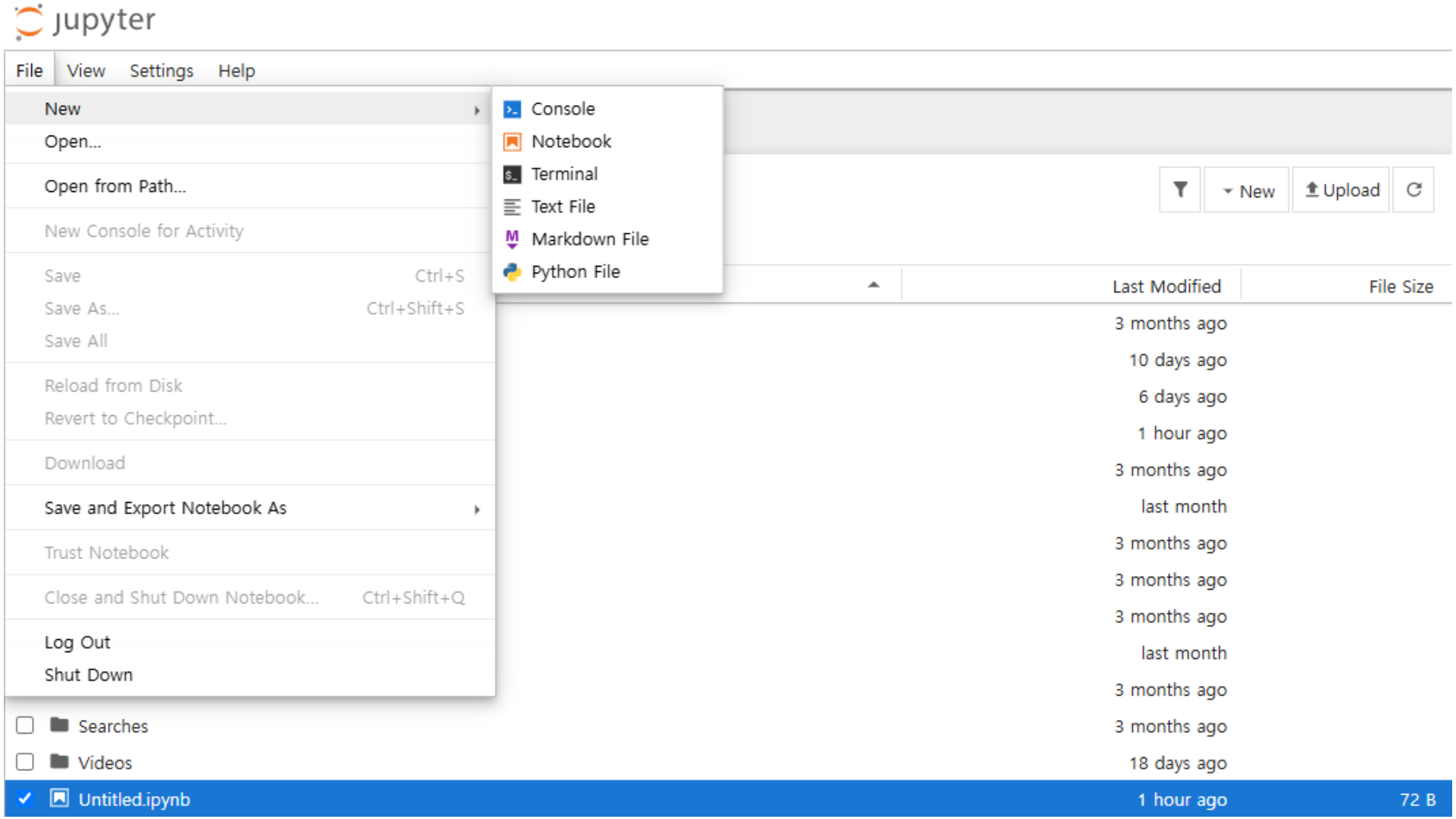
Jupyter Notebook

VS Code에서 Notebook 실행 환경 준비하기



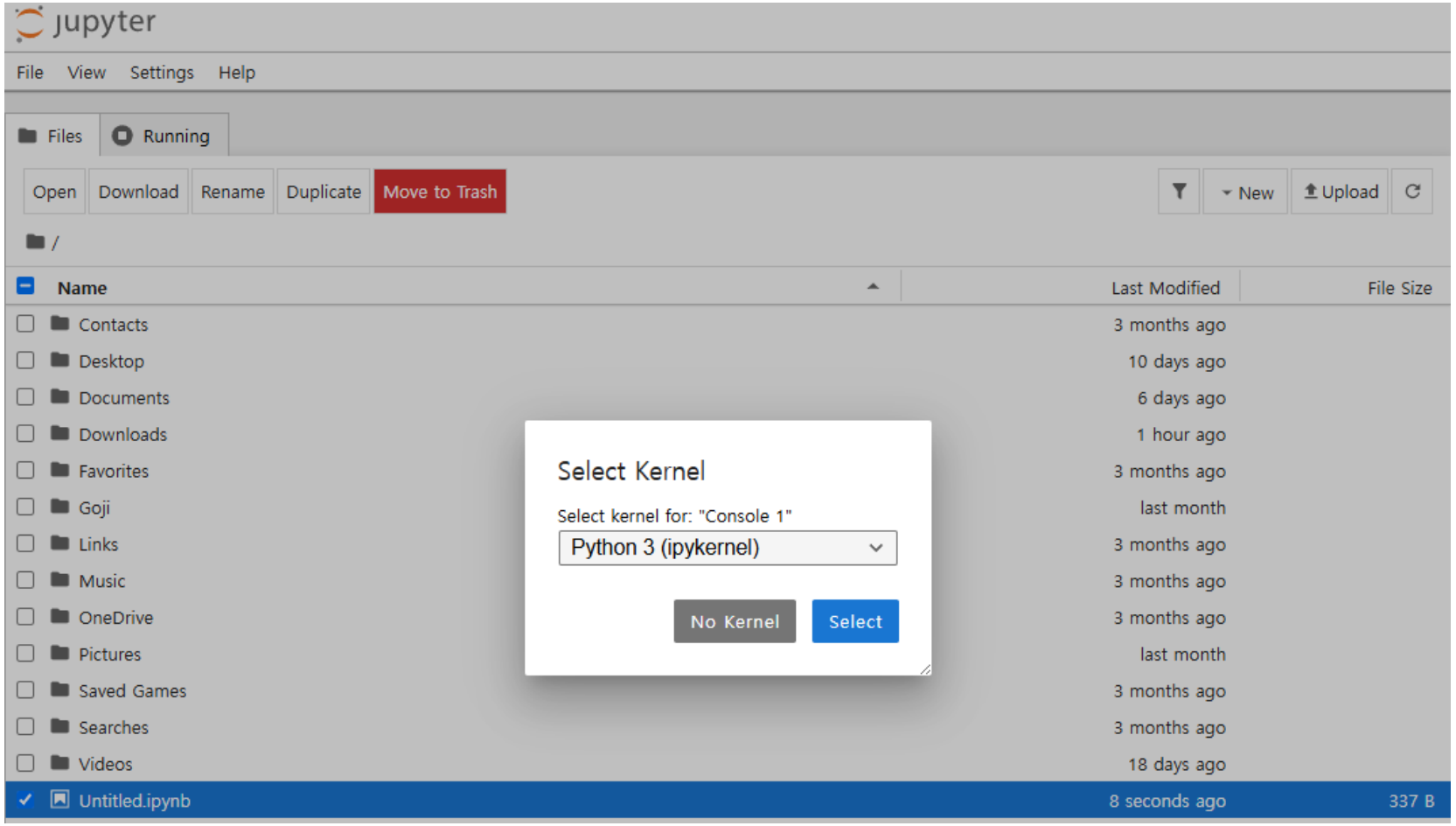
Jupyter Notebook

VS Code에서 Notebook 실행 환경 준비하기



Jupyter Notebook

커널을 선택하는 화면과 실행 환경 확인



브라우저 JupyterLab의 커널 선택 창이다. VS Code에서는 앞 장에서 설명한 Select Kernel을 사용한다. 화면의 메뉴 위치를 그대로 외우기보다, 어떤 Python 커널을 선택했는지 확인한다.

Jupyter Notebook

셀 실행, 결과 표시, 저장을 구분하기

셀 실행은 코드에 작업을 시킨다

Run Cell 버튼이나 Shift+Enter로 셀을 실행한다. Shift+Enter는 실행 후 다음 셀로 이동하며, Ctrl+Enter는 현재 셀을 실행한다.

계산값의 자동 표시를 이해한다

일반적인 Python 코드 셀은 마지막 표현식의 값을 자동으로 표시할 수 있다. 여러 결과를 명확하게 보이려면 print()를 사용한다. 대입문 자체는 출력하지 않는다.

저장과 실행은 같은 동작이 아니다

파일을 저장했다고 코드를 다시 실행한 것은 아니다. 코드를 수정한 뒤 셀을 실행하지 않으면 이전 실행 결과가 화면에 남아 있을 수 있다.

마지막 표현식을 표시하는 셀

```
x = 3
x + 4
```

표시되는 값

```
7
```

두 값을 명시적으로 출력

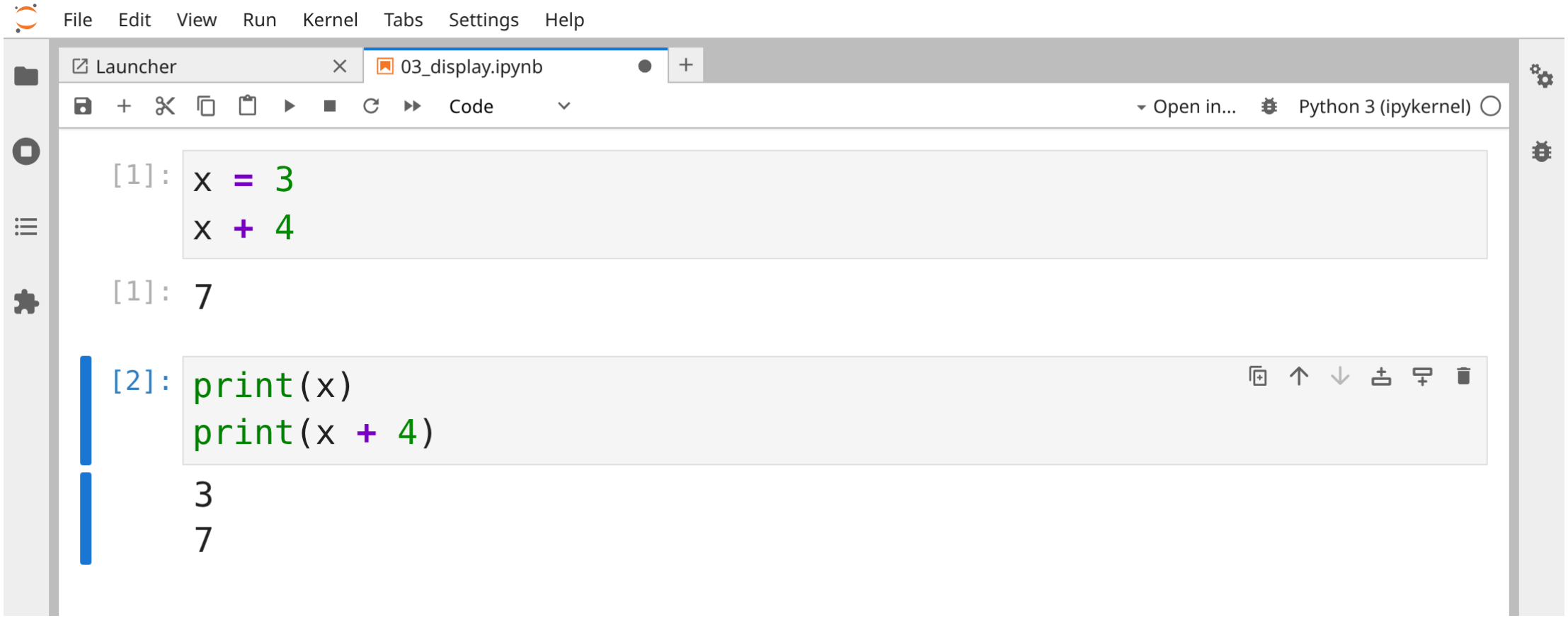
```
print(x)
print(x + 4)
```

실행 결과

```
3
7
```

Jupyter Notebook

자동 표시와 print()의 결과가 나타나는 위치



첫 셀은 마지막 표현식 x + 4의 값 7을 표시한다. 두 번째 셀은 두 print()의 결과를 모두 출력한다.
코드 입력 영역과 출력 영역을 구분하고, 어느 문장이 각 결과를 만들었는지 연결한다.

Jupyter Notebook

화면 순서와 실제 실행 순서는 다를 수 있다

변수는 커널의 현재 상태를 따른다

A 셀에서 x = 10을 실행한 뒤 B 셀에서 print(x)를 실행하면 10이 나온다. 이후 다른 셀에서 x = 20을 실행하면 B 셀의 결과도 다음 실행 때 달라진다.

위에 있는 셀이 항상 먼저 실행되지는 않는다

사용자가 셀을 선택하여 실행하므로 화면의 위아래 배치만으로 상태를 판단할 수 없다. 실행 번호와 어떤 대입을 마지막으로 실행했는지 확인한다.

처음부터 실행되는지 다시 확인한다

커널을 재시작하면 메모리의 변수 상태가 초기화된다. 문서를 저장한 뒤 Restart와 Run All로 위에서 아래까지 실행하여 빠진 초기화가 없는지 점검한다.

A 셀

```
x = 10
```

B 셀

```
print(x)
```

C 셀

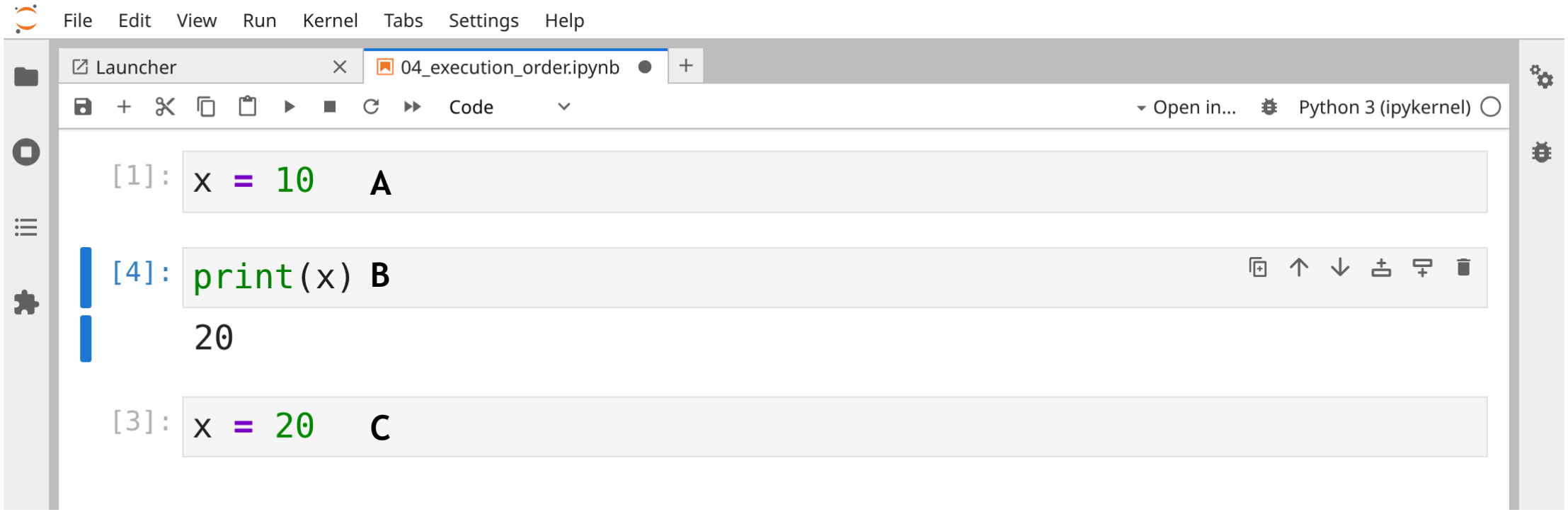
```
x = 20
```

실행 순서를 바꾸면

A → B이면 10이 출력된다.
A → C → B이면 20이 출력된다.
재시작 후 B만 실행하면 x가 없어 NameError가 발생할 수 있다.

Jupyter Notebook

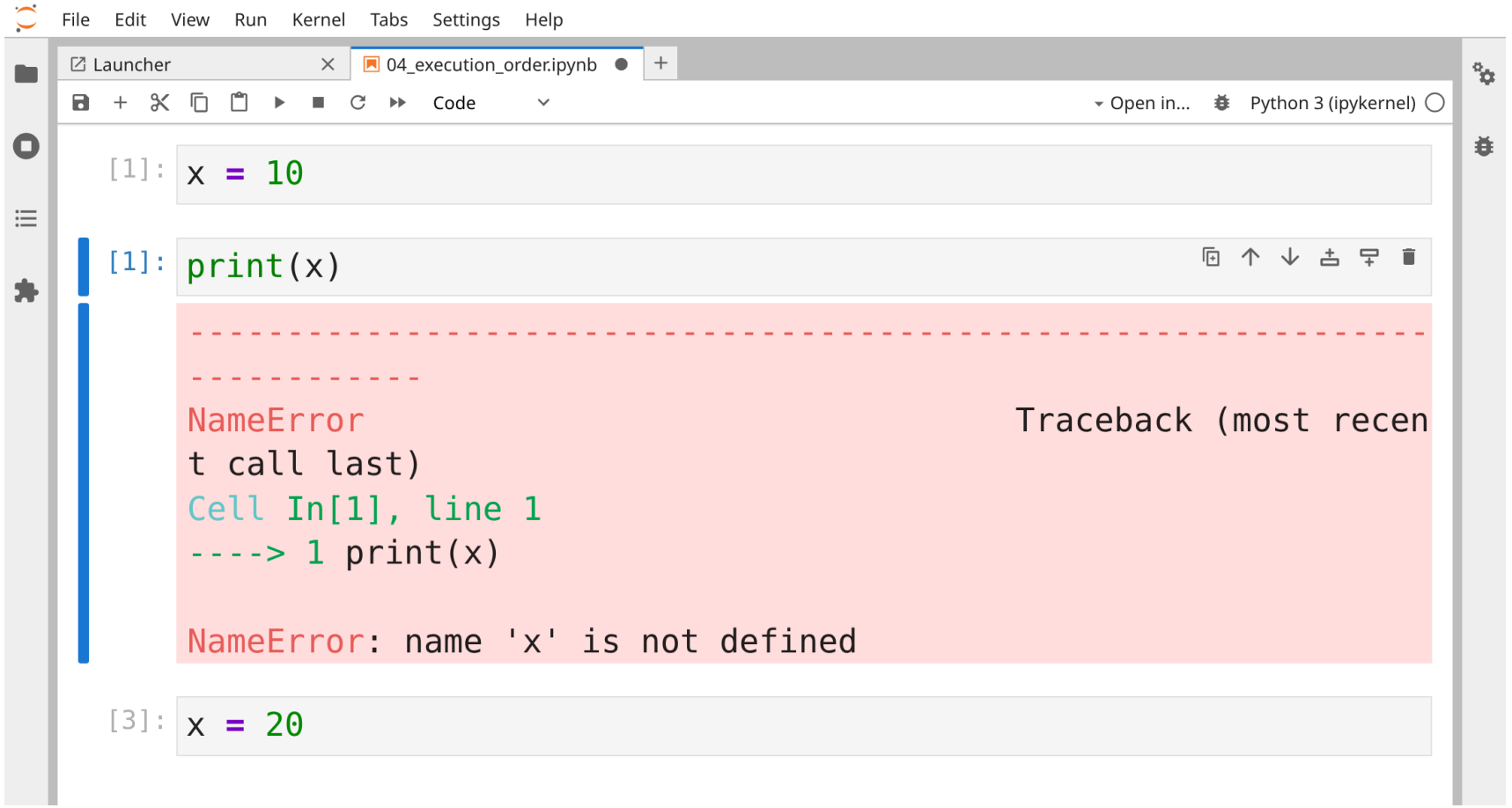
실행 번호로 셀의 실제 실행 순서 확인하기



A 셀의 x = 10 이후 C 셀의 x = 20을 실행하고 B 셀을 다시 실행한 화면이다.
B 셀의 실행 번호가 더 크며 결과는 20이다. 화면의 위아래 순서가 실행 순서를 뜻하지는 않는다.

Jupyter Notebook

커널 재시작 후에는 코드가 보여도 변수가 없을 수 있다



커널을 재시작한 뒤 print(x) 셀만 실행하여 NameError가 발생한 화면이다.
x = 10을 다시 실행한 뒤 출력 셀을 실행해야 한다. 남아 있는 코드와 실행 번호만으로 판단하지 않는다.

Jupyter Notebook

실습: 셀과 상태를 직접 확인하기

두 개의 셀을 순서대로 실행한다

첫 번째 셀에서 score를 정의하고 두 번째 셀에서 Score + 5를 출력한다. 실행 전 예상 결과를 적고, 실제 결과와 비교한다.

값을 수정하고 필요한 셀을 다시 실행한다

첫 셀의 80을 90으로 바꾼다. 편집만 한 상태와 첫 셀을 다시 실행한 상태에서 두 번째 셀의 결과가 어떻게 달라지는지 확인한다.

확인해야 할 설명

편집만으로는 커널의 score가 바뀌지 않는다. 첫 셀을 다시 실행해야 새 값이 저장되고, 두 번째 셀을 다시 실행해야 새 출력이 나타난다.

첫 번째 셀

```
score = 80
```

두 번째 셀

```
print(score + 5)
```

결과를 비교하기

처음에는 85가 출력된다.
편집만 했다면 여전히 85일 수 있다.
90을 대입한 셀까지 다시 실행하면 95가 출력된다.

Jupyter Notebook

코드를 편집한 상태와 다시 실행한 상태 비교하기

90으로 편집만 한 뒤 출력 셀 실행

```
•[1…  score = 90

[3]:  print(score + 5)
      85
```

대입 셀과 출력 셀을 차례로 다시 실행

```
[4]:  score = 90

[5]:  print(score + 5)
      95
```

첫 화면은 코드에 90이 보이지만 커널에는 80이 남아 있어 85가 출력된다.
두 번째 화면은 대입 셀까지 다시 실행했으므로 score가 90이 되고 결과도 95로 바뀐다.

Standard Input and Output

input()으로 사용자에게 값 받기

안내 문구와 입력 값을 구분한다

input("이름: ")에서 이름: 은 사용자에게 보여 주는 안내 문구이다. 반환되는 값은 사용자가 실제로 입력한 문자열이며 문구 자체는 포함되지 않는다.

입력을 마친 뒤 다음 문장으로 진행한다

input()은 입력을 기다린다. Enter를 눌러 한 줄 입력을 마치면 반환된 문자열을 왼쪽 이름에 대입하고 다음 코드를 실행한다.

출력과 입력을 나누어 쓸 수도 있다

안내 문구를 print()로 출력하고 다음 줄에서 input()을 호출할 수도 있다. 두 형태 모두 입력 결과가 어느 변수에 들어가는지 확인한다.

안내 문구를 함께 작성

```
name = input("이름: ")
print("Hi", name)
```

안내 문구를 따로 출력

```
print("Enter your name:")
somebody = input()
print("Hi", somebody)
```

입력이 끝나는 시점

코드가 멈춘 것처럼 보일 때는 input()이 사용자의 입력을 기다리는 중인지 먼저 확인한다.

Standard Input and Output

input()의 입력 대기 화면과 입력 완료 화면

사용자의 입력을 기다리는 상태

```
[*]: name = input("이름: ")
     print("Hi", name)
```

```
이름:  ↑↓ for history. Search hist
```

영재를 입력하고 Enter를 누른 뒤

```
[1]: name = input("이름: ")
     print("Hi", name)
```

```
이름:  영재
Hi 영재
```

브라우저 JupyterLab에서는 이 예시처럼 셀 아래 입력란에 값을 입력한다.
입력을 마쳐야 print()로 진행한다. VS Code에서는 입력 UI의 위치가 다를 수 있다.

Standard Input and Output

숫자를 입력해도 처음에는 문자열이다

input()의 반환 자료형은 str이다

사용자가 20을 입력하더라도 input()은 정수 20이 아니라 문자열 "20"을 반환한다. 화면에 숫자처럼 보이는 것과 실제 자료형은 구분해야 한다.

문자열의 덧셈은 연결이다

"20" + "1"은 "201"이 된다. 수치 계산을 원한다면 문자열을 숫자로 변환해야 한다. 문자열에 정수 1을 그대로 더하면 TypeError가 발생한다.

입력 직후 자료형을 확인한다

print(age)만 보면 따옴표가 표시되지 않아 정수처럼 보일 수 있다. type(age)를 함께 확인하면 계산 전에 필요한 변환을 판단할 수 있다.

20을 입력한 경우

```
age = input("나이: ")
print(age, type(age))
print(age + "1")
```

실행 결과

```
20 <class 'str'>
201
```

오류가 발생하는 다른 예

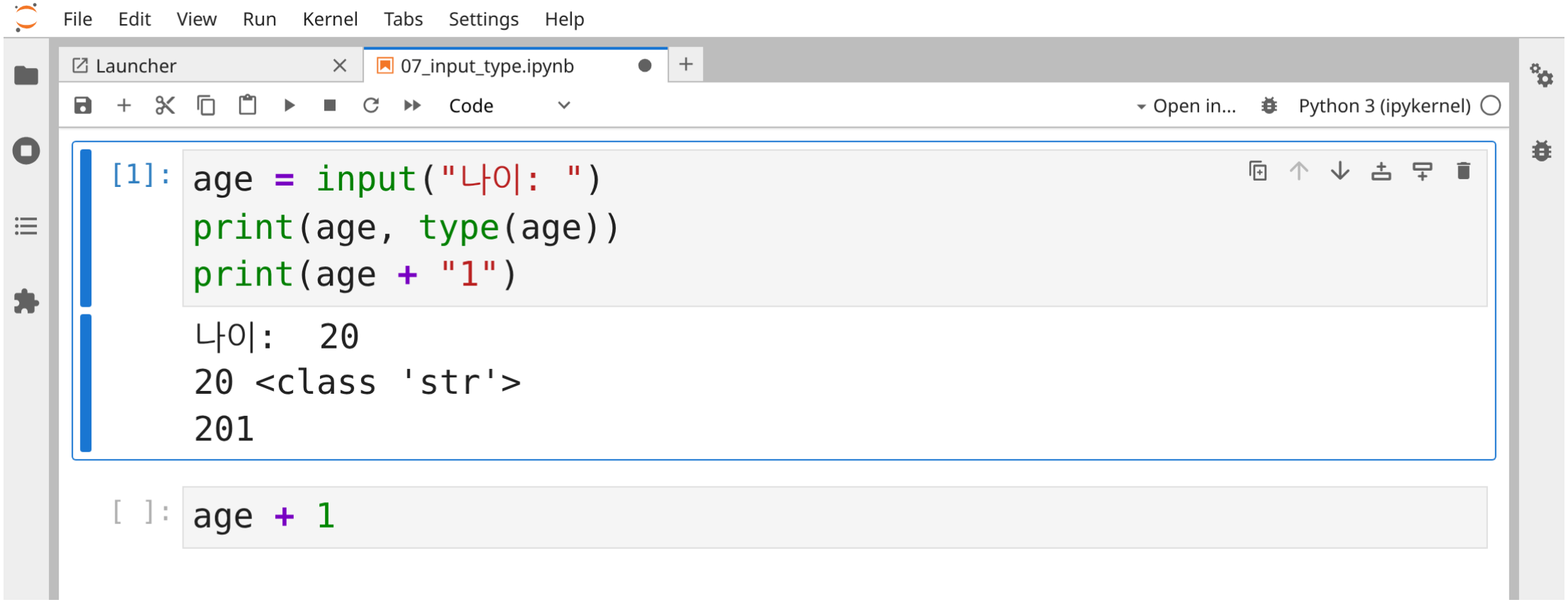
```
age + 1
```

의도에 맞게 고치기

다음 슬라이드에서는 int(age)로 수치 계산에 사용할 정수 값을 만든다.

Standard Input and Output

입력한 숫자의 자료형과 문자열 연결 결과 확인하기



20을 입력했지만 type(age)는 str이다. 따라서 age + "1"은 21이 아니라 201을 만든다.
아래의 age + 1은 아직 실행하지 않은 별도 코드이다. 실행하면 자료형이 맞지 않아 오류가 발생한다.

Type Casting

int()와 float()로 계산할 값 만들기

정수 입력에는 int()를 사용한다

int("20")은 문자열을 정수로 변환한다.
int(input(...))은 안쪽 input()으로 값을 받은 뒤 그 문자열에 int()를 적용하는 순서로 실행된다.

소수 입력에는 float()를 사용한다

float("23.5")는 부동소수점 수를 만든다. 소수점이 있는 입력을 받을 필요가 있다면 int() 대신 float()를 사용하는 것이 목적에 맞다.

변환 전후를 구분하여 읽는다

변환 함수를 호출했다고 기존 이름이 자동으로 바뀌는 것은 아니다. 반환된 값을 어느 이름에 대입했는지 확인해야 한다.

두 단계로 나누어 작성

```
text = input("나이: ")
age = int(text)
print(age + 1)
```

같은 과정을 한 문장으로 작성

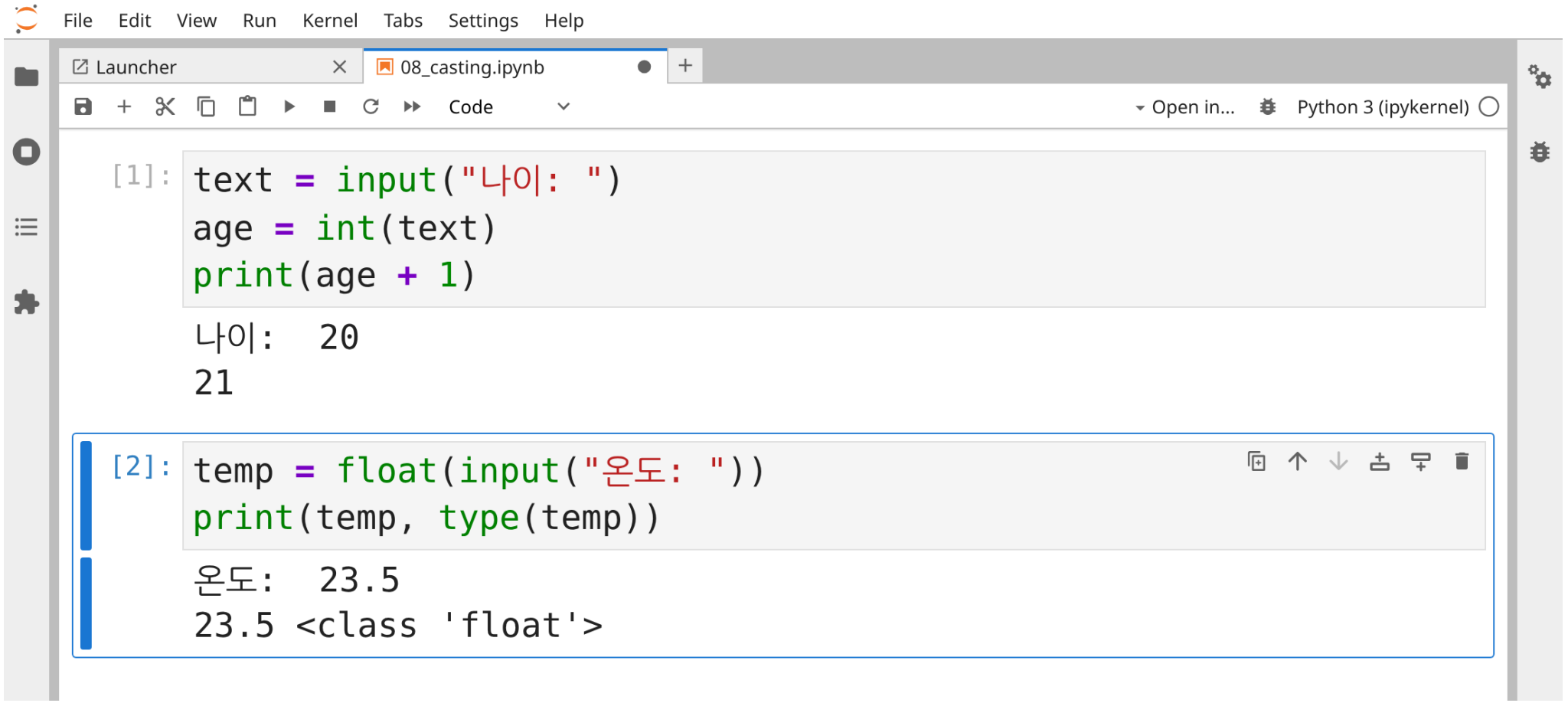
```
age = int(input("나이: "))
```

소수 입력을 받는 예

```
temp = float(input("온도: "))
print(temp, type(temp))
```

Type Casting

변환한 값을 Notebook에서 직접 계산하기



첫 셀은 입력한 "20"을 int()로 바꾸어 21을 계산한다. 두 번째 셀은 23.5를 float로 변환한다.
각 셀의 입력값, 변환 함수, 출력된 값과 자료형을 차례로 확인한다.

Type Casting

변환할 수 없는 입력은 ValueError를 만든다

정수로 읽을 수 있는 형식이 필요하다

int("20")은 가능하지만 int("20.5")나 int("스물")은 실패한다. 문자열에 숫자가 일부 들어 있다는 사실만으로 정수 변환이 가능한 것은 아니다.

형식 오류와 값의 범위는 다르다

int("-3")은 변환 자체는 성공한다. 그러나 사람의 나이로 사용할 수 있는지는 별도로 판단해야 한다. 형식변환에 성공했다고 입력의 의미까지 타당한 것은 아니다.

입력 조건을 미리 설명한다

현재 예제는 안내에 맞는 숫자를 입력한다고 가정한다. 예외 처리 문법은 뒤에서 배우고, 지금은 오류가 입력·변환·계산 중 어느 단계인지 구분한다.

변환 결과 비교

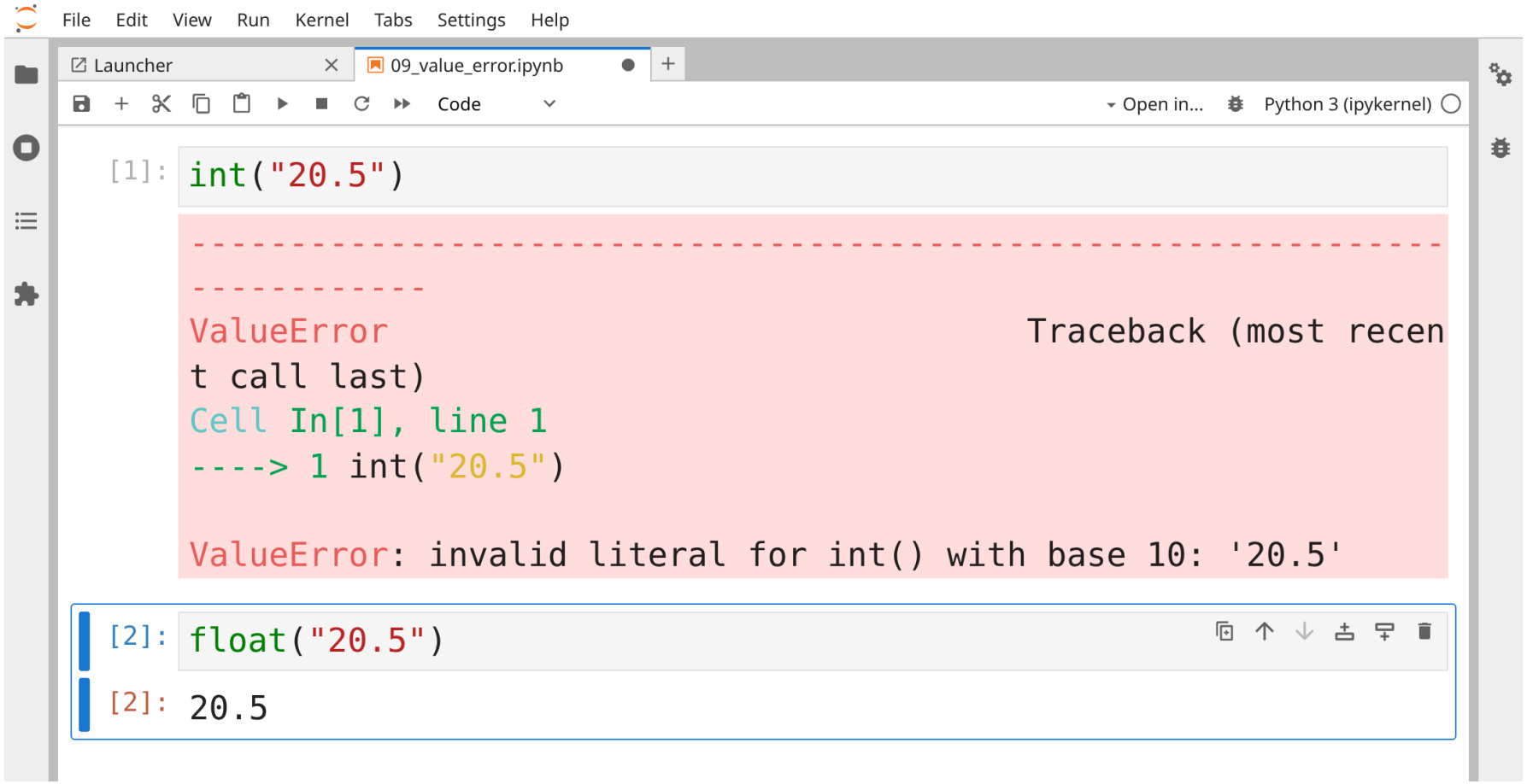
표현식	결과
int("20")	20
int("-3")	-3
int("20.5")	ValueError
float("20.5")	20.5
int("")	ValueError

자주 하는 잘못된 수정

소수를 정수로 억지로 바꾸기보다 어떤 입력을 받으려는지 먼저 정한다. 빈 입력도 숫자로 변환되지 않는다.

Type Casting

변환 오류와 정상 결과를 셀별로 비교하기



int("20.5")는 ValueError를 내지만 float("20.5")는 20.5를 만든다.
오류가 난 첫 셀과 정상 실행한 두 번째 셀을 구분하고, 원하는 입력 형식에 맞는 함수를 선택한다.

Type Casting

예제: 입력한 나이로 남은 기간 계산하기

계산에 사용할 정수로 변환한다

사용자가 입력한 문자열에 int()를 적용하여 age에 정수를 연결한다. 따라서 65 - age는 문자열 연산이 아니라 정수 뺄셈이 된다.

입력과 출력의 대응을 확인한다

53을 입력하면 age는 53이고 65 - 53은 12이다. 첫 출력은 입력받은 나이를, 두 번째 출력은 기준값까지 남은 기간을 보여 준다.

계산의 전제도 읽어야 한다

이 예제의 65는 계산을 위한 가정이며 실제 은퇴·정년 규정을 뜻하지 않는다. 70을 입력하면 -5가 나오므로 음수 결과를 어떻게 해석할지도 생각한다.

예제 코드

```
age = int(input("How old are you? "))
print("Your age is", age)
print(65 - age, "years to retirement")
```

53을 입력한 실행 예

```
How old are you? 53
Your age is 53
12 years to retirement
```

다른 값을 넣어 보기

65를 입력하면 0, 70을 입력하면 -5가 계산된다. 실행 성공과 의미 있는 결과를 구분한다.

Standard Input and Output

print()에서 값, 공백, 줄바꿈 구분하기

여러 값은 기본적으로 공백으로 구분한다

print("Hello", "Again")은 두 문자열 사이에 공백을 넣어 출력한다. 쉼표는 출력할 값을 나누는 문법이며 쉼표 자체가 화면에 나타나는 것은 아니다.

아무 값도 주지 않으면 빈 줄을 만든다

print()의 기본 동작은 출력 마지막에 줄바꿈을 붙이는 것이다. 따라서 인수가 없는 print()는 내용 없이 줄만 바꾼다.

출력 모양과 실제 값을 구분한다

sep은 값 사이의 구분 문자열, end는 출력 마지막에 붙일 문자열을 지정한다. 출력 형식을 바꾸어도 변수에 저장된 값 자체가 변하는 것은 아니다.

출력 형태 비교

```
print("Hello", "Again")
print()
print("A", "B", sep="-")
print("Hello", end=" ")
print("Python")
```

실행 결과

```
Hello Again

A-B
Hello Python
```

Practice

실습: 두 수를 입력받아 합계와 평균 구하기

입력과 계산 단계를 먼저 적는다

두 수를 각각 입력받고 소수를 처리할 수 있는 자료형으로 변환한다. 합계는 두 값을 더하고, 평균은 합계를 2로 나누어 계산한다.

주어진 입력으로 결과를 예상한다

첫 수가 10이고 두 번째 수가 15라면 합계는 25, 평균은12.5이다. 계산 전에 문자열 연결이 일어나지 않도록 변환 위치를 확인한다.

프로그램이 가져야 할 동작

두 입력을 한 번씩 받고 합계와 평균을 각각 출력한다. 입력을 3.5와 4.5로 바꾸었을 때도 같은 코드를 수정 없이 사용할 수 있어야 한다.

완성할 코드

```
a = _____(input("첫 수: "))
b = _____(input("두 번째 수: "))
total = _____
average = _____
print("합계:", total)
print("평균:", average)
```

점검 질문

input()은 어떤 자료형을 반환하는가?
평균 계산에 어떤 괄호가 필요한가?
예상한 값과 실제 출력은 같은가?

Practice

해설: 문자열을 변환한 뒤 계산한다

숫자를 받을 때 float()를 선택한다

이 실습은 정수뿐 아니라 3.5 같은 입력도 허용한다. float()로 변환하면 이후의 +는 수치 덧셈으로 작동한다.

중간 결과에 이름을 붙인다

total에는 a + b의 결과를, average에는 total / 2의 결과를 연결한다. 식을 나누면 계산 단계를 확인하기 쉽고 중간 값을 출력하여 점검할 수 있다.

잘못된 결과의 원인을 비교한다

변환 없이 "10"과 "15"를 더하면 "1015"가 된다. 또한 a + b / 2는 (a + b) / 2와 다른 식이다. 자료형과 괄호를 함께 확인한다.

완성 코드

```
a = float(input("첫 수: "))
b = float(input("두 번째 수: "))
total = a + b
average = total / 2
print("합계:", total)
print("평균:", average)
```

10과 15를 입력한 결과

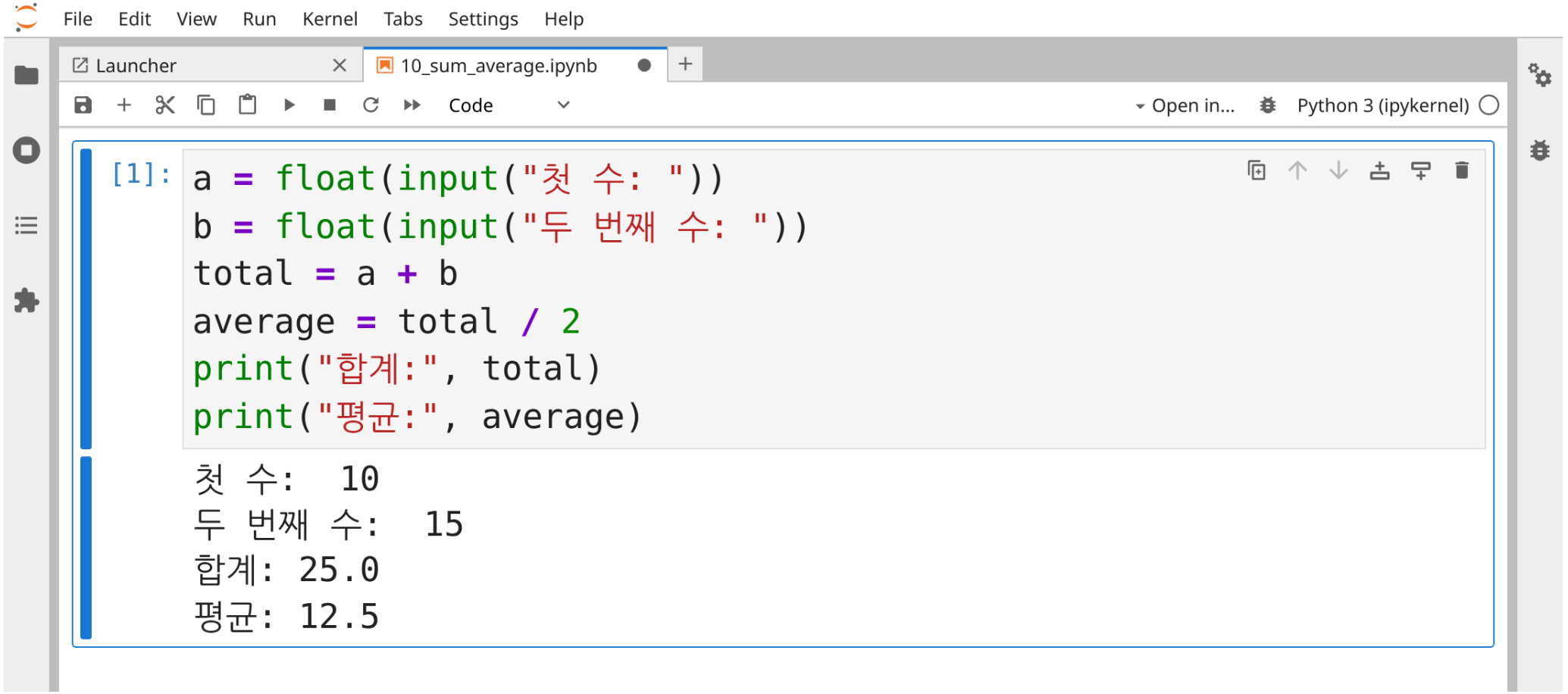
합계: 25.0
평균: 12.5

3.5와 4.5를 입력한 결과

합계: 8.0
평균: 4.0

Practice

두 수를 입력받아 합계와 평균을 계산한 화면



10과 15를 입력하면 float 변환을 거쳐 합계 25.0과 평균 12.5가 출력된다.
입력 두 줄, 계산 두 줄, 출력 두 줄을 화면의 결과와 연결하여 읽는다.

Basic Operators

산술 연산자는 수치 계산의 기본 도구이다

a + b에서 +는 수행할 연산이고 a와 b는 연산에 사용할 값이다. 같은 식도 a와 b의 값이나 자료형이 달라지면 결과가 달라질 수 있다.

연산자	의미	a = 10, b = 20일 때	결과
+	덧셈	a + b	30
-	뺄셈	a - b	-10
*	곱셈	a * b	200
/	나눗셈	b / a	2.0
//	내림 나눗셈	b // a	2
%	나머지	b % a	0
**	거듭제곱	a ** b	100000000000000000000

결과의 자료형도 확인한다

Python에서 정수끼리 /로 나누어도 결과는 float이다. b / a의 2.0과 b // a의 2는 수치가 같아도 자료형이 다르다.

Basic Operators

나눗셈, 내림 나눗셈, 나머지는 목적이 다르다

/는 나눈 값을 구한다

7 / 4는 1.75이다. 정수 몫만 필요한 상황과 전체 비율을 계산해야 하는 상황을 구분한다. 평균이나 온도 변환에는 보통 /가 필요하다.

//와 %는 묶음과 남는 양을 구분한다

양의 정수에서 7 // 4는 완성된 묶음 수 1, 7 % 4는 남은 수 3이다. 7 = 4 × 1 + 3으로 계산 결과를 확인할 수 있다.

나누는 값은 0일 수 없다

분모가 코드에 직접 적혀 있지 않아도 실행 시점에 0이면 문제가 생긴다. /, //, % 모두 나누는 값에 대한 조건을 확인해야 한다.

같은 두 값으로 비교

```
print(7 / 4)
print(7 // 4)
print(7 % 4)
```

실행 결과

```
1.75
1
3
```

실생활 계산에 대응

물건 7개를 한 묶음에 4개씩 담으면 완성된 묶음은 1개이고 남은 물건은 3개이다.

Basic Operators

예제: 분자와 분모를 입력받기

입력한 두 정수를 서로 다른 이름에 둔다

p는 분자, q는 분모를 나타낸다. input()으로 받은 두 값을 각각 int()로 변환하므로 이후 //와 %는 정수 계산에 사용된다.

같은 입력에 두 연산을 적용한다

p // q와 p % q는 같은 나눗셈을 서로 다른 관점에서 결과이다. q가 0이 아니라면 정수 계산에서 p = (p // q) × q + (p % q)가 성립한다.

입력 조건을 바꾸어 확인한다

7과 4뿐 아니라 8과 4, 3과 4도 넣어 본다. 나누어떨어지는 경우에는 나머지가 0이고, 분자가 더 작으면 양수의 정수 몫은 0이다.

예제 코드

```
p = int(input("분자: "))
q = int(input("분모: "))
print("몫 =", p // q)
print("나머지 =", p % q)
```

7과 4를 입력한 결과

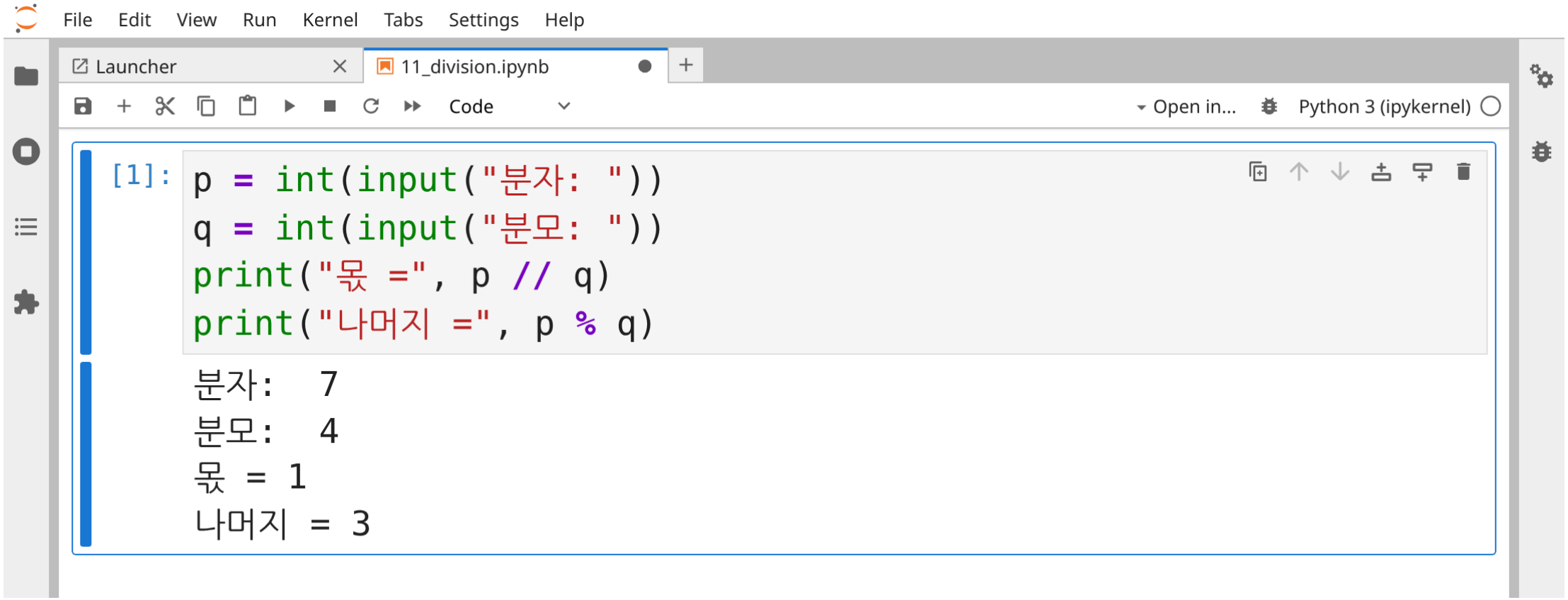
```
몫 = 1
나머지 = 3
```

3과 4를 입력한 결과

```
몫 = 0
나머지 = 3
```

Basic Operators

분자와 분모를 입력하고 몫과 나머지 확인하기



분자에 7, 분모에 4를 입력하여 같은 두 정수에 //와 %를 각각 적용한 화면이다.
몫 1과 나머지 3을 이용해 7 = 4 × 1 + 3이 되는지 다시 확인한다.

Basic Operators

음수에서는 //를 단순한 소수점 제거로 보지 않는다

내림의 방향은 음의 무한대이다

-7 / 4는 -1.75이다. 이 값을 아래쪽 정수로 내리면 -2이므로 -7 // 4는 -2이다. 0에 가까운 쪽으로 잘라 내는 동작과 다르다.

나머지도 같은 나눗셈 관계를 만족한다

-7 = 4 × (-2) + 1이므로 -7 % 4는 1이다. 양수에서 외운 몫과 나머지의 모양을 음수에 기계적으로 적용하지 않는다.

다른 연산과 비교해 의미를 확인한다

int(-1.75)는 0 방향으로 소수 부분을 버려 -1이 된다. int()를 적용하는 것과 //를 사용하는 것은 음수에서 다른 결과를 만들 수 있다.

결과를 직접 비교

```
print(-7 / 4)
print(-7 // 4)
print(-7 % 4)
print(int(-1.75))
```

실행 결과

```
-1.75
-2
1
-1
```

한 문장으로 구분

//는 내림 나눗셈이다. int()의 수치 변환과 같은 연산이라고 가정하지 않는다.

Operator Precedence

괄호는 계산 의도를 드러낸다

곱셈은 덧셈보다 먼저 묶인다

x + y * z는 x + (y * z)로 해석된다. x, y, z가 각각 2, 3, 4이면 곱셈 결과 12에 2를 더하여 14가 된다.

괄호를 추가하면 먼저 계산할 부분이 달라진다

(x + y) * z에서는 2 + 3을 먼저 계산한다. 같은 세 값과 기호를 사용해도 괄호에 따라 20이라는 다른 결과가 나온다.

같은 우선순위에서도 묶이는 방향을 확인한다

10 - 3 - 2는 왼쪽부터 묶여 5이다. 거듭제곱은 오른쪽부터 묶이므로 2 ** 3 ** 2는 2 ** 9, 즉 512이다.

두 식 비교

```
x, y, z = 2, 3, 4
print(x + y * z)
print((x + y) * z)
```

실행 결과

```
14
20
```

거듭제곱 비교

```
print(2 ** 3 ** 2)
print((2 ** 3) ** 2)
```

실행 결과

```
512
64
```

Practice

실습: 화씨 온도를 섭씨 온도로 변환하기

수학 식을 Python 표현식으로 옮긴다

화씨 온도 F를 섭씨 온도 C로 바꾸는 식은
C = (F - 32) × 5 / 9이다.
먼저 32를 빼고 그 결과에 비율 5 / 9를 곱한다.

입력, 처리, 출력을 분리한다

정수 화씨 온도를 입력받아 ftemp에 저장하고 ctemp를 계산한다. 계산 결과는 소수가 될 수 있으므로 나눗셈 기호를 신중하게 선택한다.

알려진 값으로 계산을 점검한다

100을 입력하면 약 37.78이 되어야 한다. 32를 넣었을 때 0이 나오는지도 확인하면 괄호나 뺄셈 순서의 실수를 찾는 데 도움이 된다.

완성할 식

```
ftemp = int(input("화씨온도: "))
ctemp = _____________________
print("섭씨온도:", ctemp)
```

생각할 질문

왜 ftemp - 32를 괄호로 묶는가?
/ 대신 //를 사용하면 무엇이 달라지는가?
100.5를 입력할 수 있는 코드인가?

Practice

해설: 식의 순서와 입력 조건 확인하기

정수 입력 예제의 실행 과정

100을 입력하면 ftemp는 100이다. 100 - 32.0은 68.0, 여기에 5.0을 곱하면 340.0이고 9.0으로 나누면 약 37.78이 된다.

괄호를 빼면 다른 식이 된다

ftemp - 32.0 * 5.0 / 9.0은 32.0에 대한 곱셈과 나눗셈을 먼저 한다. 원래 변환식과 계산 순서가 다르므로 같은 프로그램으로 볼 수 없다.

소수 입력을 허용하는 변형

입력 줄의 int()를 float()로 바꾸면 100.5 같은 값을 받을 수 있다. 이 변경은 출력만 꾸미는 것이 아니라 프로그램이 받아들이는 입력 범위를 넓힌다.

정수 입력을 받는 예제

```
ftemp = int(input("화씨온도: "))
ctemp = (ftemp - 32.0) * 5.0 / 9.0
print("섭씨온도:", ctemp)
```

100을 입력한 결과

섭씨온도: 37.77777777777778

소수 입력을 받는 변형

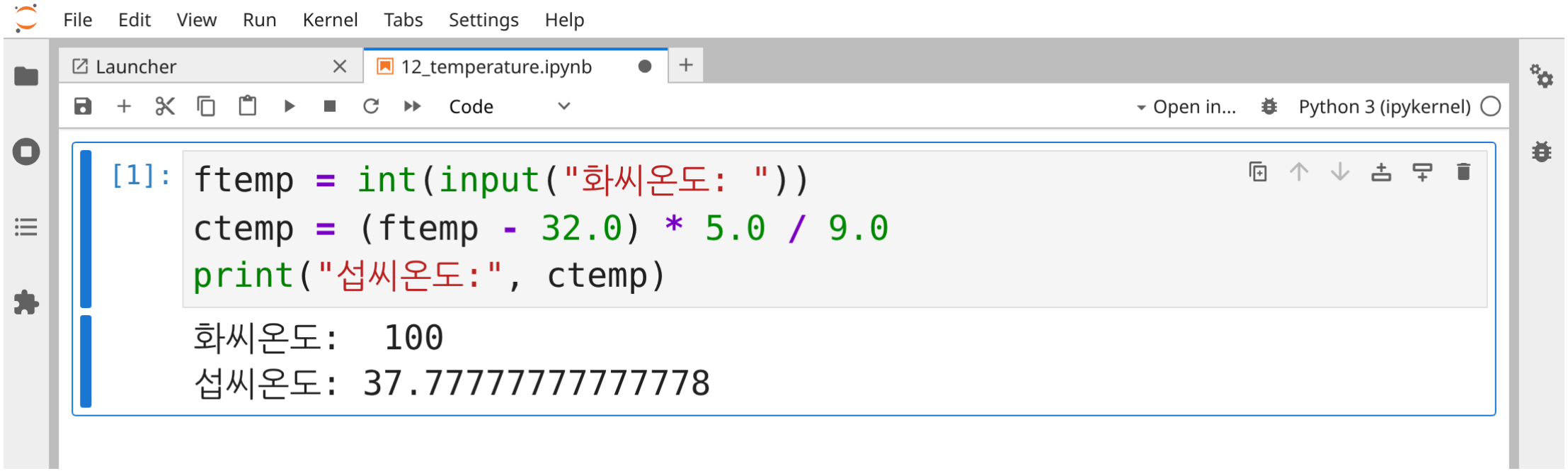
```
ftemp = float(input("화씨온도: "))
```

같이 확인할 입력

32이면 0.0, 212이면 100.0이다. 변형 예제의 100.5는 약 38.06이다.

Practice

화씨 온도 변환 프로그램의 실제 입력과 출력



화씨온도에 100을 입력하면 섭씨온도는 37.77777777777778로 출력된다.
결과만 읽지 말고 입력값 100이 괄호 안의 뺄셈과 이후 곱셈·나눗셈에 어떻게 쓰였는지 설명한다.

Comparison Operators

비교 결과는 조건 판단에 사용한다

비교는 값을 새로 대입하는 동작이 아니다. a와 b를 비교해도 두 이름이 가리키는 값은 그대로 유지된다. 여기서는 기본 숫자의 비교 결과인 True와 False를 다룬다.

연산자	의미	a = 10, b = 20일 때	결과
==	같다	a == b	False
!=	다르다	a != b	True
>	크다	a > b	False
<	작다	a < b	True
>=	크거나 같다	a >= b	False
<=	작거나 같다	a <= b	True

경계값을 포함할지 구분한다

score > 60과 score >= 60은 score가 60일 때 다르다. 다음 시간의 선택 구조에서는 이 차이가 실제로 실행되는 코드의 차이로 이어진다.

Logical Operators

and, or, not으로 조건 결합하기

and는 두 조건을 모두 요구한다

age >= 18 and is_student는 두 부분이 모두 참이어야 참이다. 조건을 자연어로 읽으면 “18세 이상이면서 학생이다”가 된다.

or는 적어도 하나의 참을 허용한다

age < 18 or is_student는 한 부분만 참이어도 참이다. 두 부분이 모두 참인 경우도 참이므로 “둘 중 정확히 하나만”과는 다르다.

not은 참과 거짓을 반대로 판단한다

not is_student는 학생 여부를 반대로 판단한다.
지금처럼 비교식과 bool을 조합하면 결과도 bool이지만 일반적인 and와 or는 피연산자 자체를 반환할 수 있다.

같은 값으로 세 조건 비교

```
age = 20
is_student = True
print(age >= 18 and is_student)
print(age < 18 or is_student)
print(not is_student)
```

실행 결과

```
True
True
False
```

짧게 검증하기

is_student를 False로 바꾸면 첫 결과와 두 번째 결과는 모두 False가 된다.

Membership Operators

in과 not in은 포함 여부를 확인한다

대상이 안에 있는지 묻는 연산이다

문자열에서 in은 특정 부분 문자열이 포함되는지 확인한다. "Py" in "Python"은 참이고 "py" in "Python"은 거짓이다. 대소문자는 구분된다.

not in은 포함되지 않았는지 확인한다

대상이 없으면 참이 된다. not in은 두 단어로 쓰지만 포함되지 않음을 묻는 하나의 비교 형태로 읽으면 된다.

비교 대상의 종류를 확인한다

문자열의 부분 문자열 검사와 리스트의 원소 검사는 대상이 다르다. 여기서는 문자열로 확인하고, 리스트의 자세한 사용은 자료구조 단원에서 다룬다.

문자열의 포함 여부

```
text = "Python"
print("Py" in text)
print("py" in text)
print("Java" not in text)
```

실행 결과

```
True
False
True
```

혼동하지 않기

in은 두 값이 같은지 묻는 ==와 다르다. 전체 문자열이 같지 않아도 일부가 포함될 수 있다.

Membership Operators

부분 문자열과 대소문자에 따른 실행 결과

```
[2]: text = "Python"
     print("Py" in text)
     print("py" in text)
     print("Java" not in text)
```

```
True
False
True
```

"Py"는 "Python"에 포함되지만 "py"는 포함되지 않아 앞의 두 결과가 서로 다르다.
마지막 식은 "Java"가 없다는 조건을 확인하므로 True이다. 포함 여부와 값의 같음을 구분한다.

Bitwise Operators

진법은 같은 정수를 표기하는 다른 방법이다

정수 값과 표기법을 구분한다

십진수 13은 이진수로 1101이다. Python에서는 0b1101이라고 적어 이진수 정수 리터럴임을 나타낸다. 값 자체는 십진수로 적은 13과 같다.

변환 함수는 표기 문자열을 만든다

bin(13)은 "0b1101"을, oct(13)은 "0o15"를, hex(13)은 "0xd"를 반환한다. 다른 진법으로 표시한 결과는 문자열이라는 점에 주의한다.

비트의 자리값으로 확인한다

1101은 8 + 4 + 0 + 1이다. 오른쪽부터 1, 2, 4, 8의 자리값을 대응시키면 다음 슬라이드의 비트 연산도 이해할 수 있다.

같은 값을 다른 방법으로 표시

```
print(0b1101)
print(bin(13))
print(oct(13))
print(hex(13))
```

실행 결과

```
13
0b1101
0o15
0xd
```

접두어 확인

0b는 2진수, 0o는 8진수, 0x는 16진수 표기이다.

Practice

결과를 먼저 예상하고 이유를 설명하기

각 식의 값을 먼저 예상한 뒤 실행한다. 눈에 보이는 숫자만 비교하지 말고 문자열인지 정수인지, 어떤 연산자를 사용했는지 함께 확인한다.

표현식	예상 결과를 적을 때 확인할 내용
int("7") + 4	문자열 변환이 먼저 일어나는가?
"7" + "4"	숫자 덧셈인가, 문자열 연결인가?
7 // 4, 7 % 4	몫과 나머지를 구분했는가?
2 + 3 * 4 == 14	산술 계산과 비교 중 무엇이 먼저인가?
2 ^ 3, 2 ** 3	두 연산자의 의미가 같은가?

실행 후 확인할 답

순서대로 11, "74", 1과 3, True, 1과 8이다. 틀린 항목은 값을 외우기보다 어떤 단계나 기호를 다르게 해석했는지 설명한다.

Chapter Overview

입력한 값을 계산하고 검증하는 과정

실행 환경

CLI에서는 명령문을 입력하고 Notebook에서는 셀을 실행한다. Notebook의 출력은 마지막으로 실행한 코드와 커널 상태에 의존하므로 저장과 재실행을 구분한다.

입출력과 자료형

input()이 반환한 문자열을 그대로 사용할지, int()나 float()로 변환할지 정한다.
print()로 중간 결과와 자료형을 확인하면 오류 원인을 좁힐 수 있다.

연산자와 다음 단계

산술 연산으로 값을 계산하고 비교·논리 연산으로 조건을 만든다.
다음 시간에는 그 조건에 따라 실행할 문장을 선택하고, 작업을 반복하는 제어 구조를 배운다.